

```
#include "Matrix3x3.h"
```

```
#include <cassert>
```

```
bool Matrix3x3::operator==(const Matrix3x3& aOther) const noexcept
```

```
{
    const Matrix3x3& M = *this;
    const Matrix3x3& N = aOther;

    // why this one would know to use the operator overloading in problem set 1
    return M[0] == N[0] && M[1] == N[1] && M[2] == N[2];
}
```

```
Matrix3x3 Matrix3x3::transpose() const noexcept
```

```
{
    const Matrix3x3& M = *this;

    Vector3D newRow0(M[0].x(), M[1].x(), M[2].x());
    Vector3D newRow1(M[0].y(), M[1].y(), M[2].y());
    Vector3D newRow2(M[0].w(), M[1].w(), M[2].w());

    return Matrix3x3(newRow0, newRow1, newRow2);
}
```

```
Matrix3x3 Matrix3x3::operator*(const Matrix3x3& aOther) const noexcept
```

```
{
    const Matrix3x3& F = *this;
    const Matrix3x3& G = aOther;

    Vector3D col0 = G.column(0);
    Vector3D col1 = G.column(1);
    Vector3D col2 = G.column(2);

    Vector3D row0(F[0].dot(col0), F[0].dot(col1), F[0].dot(col2));
    Vector3D row1(F[1].dot(col0), F[1].dot(col1), F[1].dot(col2));
    Vector3D row2(F[2].dot(col0), F[2].dot(col1), F[2].dot(col2));

    return Matrix3x3(row0, row1, row2);
}
```

```
float Matrix3x3::det() const noexcept
```

```
{
    const Matrix3x3& M = *this;

    float a = M[0].x();
    float b = M[0].y();
    float c = M[0].w();

    float d = M[1].x();
    float e = M[1].y();
    float f = M[1].w();
}
```

```
float g = M[2].x();
float h = M[2].y();
float i = M[2].w();

// a(ei - fh) - b(di - fg) + c(dh - eg)
return a * (e * i - f * h)
       - b * (d * i - f * g)
       + c * (d * h - e * g);
}

bool Matrix3x3::hasInverse() const noexcept
{
    const float epsilon = 0.000001f;
    return std::abs(det()) > epsilon;
}

Matrix3x3 Matrix3x3::inverse() const noexcept
{
    const Matrix3x3& M = *this;

    float a = M[0].x();
    float b = M[0].y();
    float c = M[0].w();

    float d = M[1].x();
    float e = M[1].y();
    float f = M[1].w();

    float g = M[2].x();
    float h = M[2].y();
    float i = M[2].w();

    float A = (e * i - f * h);
    float B = (d * i - f * g);
    float C = (d * h - e * g);

    float D = (b * i - c * h);
    float E = (a * i - c * g);
    float F = (a * h - b * g);

    float G = (b * f - c * e);
    float H = (a * f - c * d);
    float I = (a * e - b * d);

    Matrix3x3 adjugate(
        Vector3D(A, -D, G),
        Vector3D(-B, E, -H),
        Vector3D(C, -F, I)
    );

    float determinant = det();
    float invDet = 1.0f / determinant;
```

```
    return adjugate * invDet;
}

std::ostream& operator<<(std::ostream& aOStream, const Matrix3x3& aMatrix)
{
    aOStream << "["
        << aMatrix[0].toString() << ", "
        << aMatrix[1].toString() << ", "
        << aMatrix[2].toString()
        << "];"

    return aOStream;
}
```